

Module-4: Expert Systems

- Design Techniques, components
- Problem & Knowledge Domain
- Knowledge Engineering Approach
- Error in design of expert system
- Life cycle of Expert System
- MYCIN and Dendral

Dr. Manas Ranjan Patra
NIST University

What is an expert system?

“An expert system is a software that emulates the problem-solving / decision-making ability of a human expert.”

It uses AI techniques to solve complex problems in a **specific domain** by reasoning with the stored knowledge.

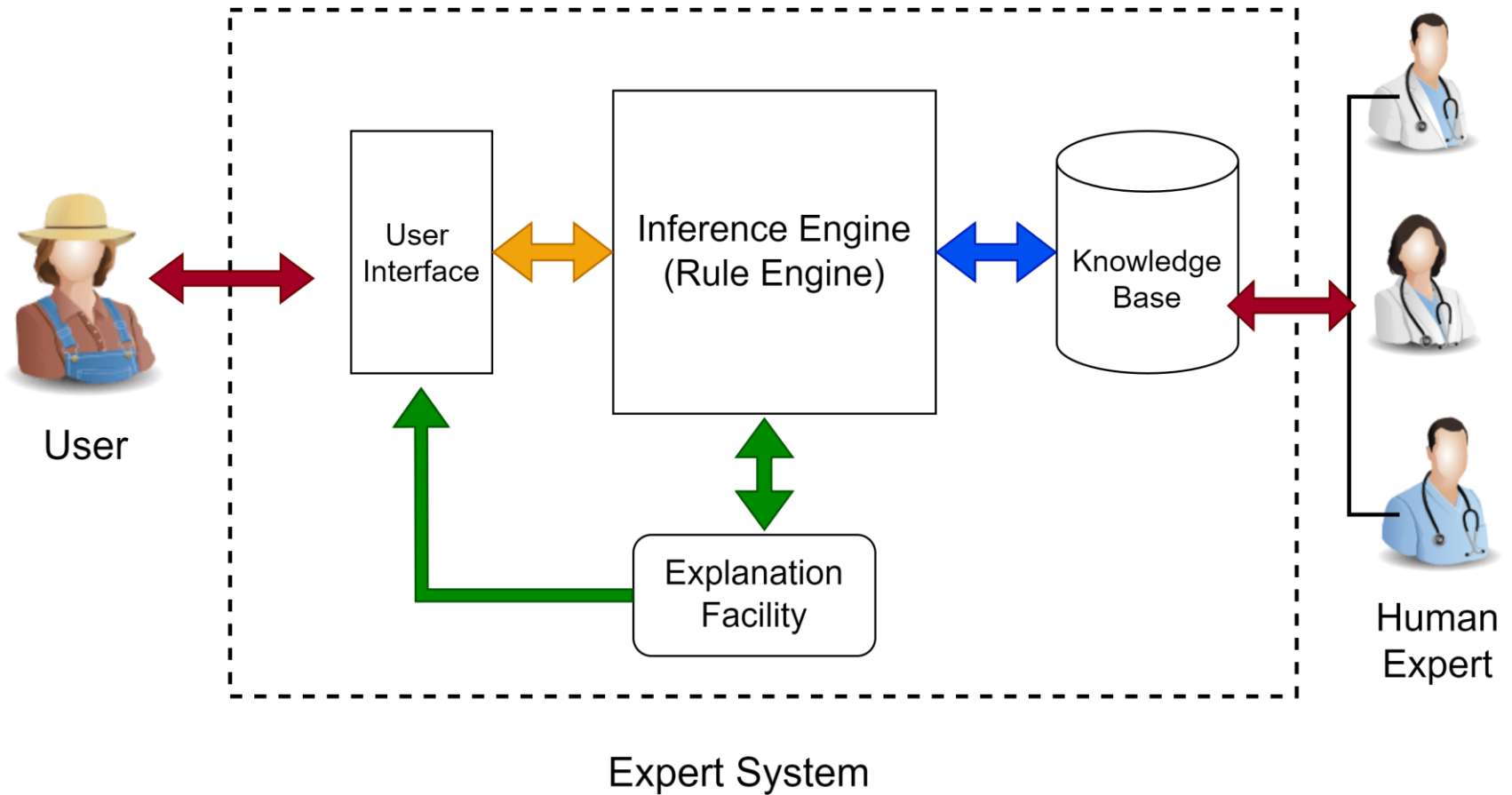
Expert systems (ES)

- ES is an application program for solving **knowledge-intensive problems**.
- ES performs specialized tasks that are possible only with human experts.
- An ES captures domain-specific knowledge & decision making capability of experts.
- ES simulates human reasoning process by applying domain specific knowledge & inferences.
- Different from traditional s/w and reflects human judgment & intuition.

Application areas

- **Medical Diagnosis:** Assisting doctors in the diagnosis of diseases (e.g., MYCIN for blood infections).
- **Engineering:** Troubleshooting (fault-finding) in machinery or engineering design
- **Finance:** Providing advice for investment.
- **Customer Support:** Responding to customer queries and resolving problems.

Expert System Design: Components



1. Knowledge Base (KB)

It contains facts and rules about a specific domain which encodes the expert's knowledge.

2. Inference Engine

This applies logical reasoning (rule-based reasoning) to the knowledge base to draw conclusions or make decisions.

3. User Interface

It facilitates user interaction with the system through which a user can input queries, and receive solutions or explanations.

4. Explanation Facility

Explains the systems reasoning process and provides explanation for the decision/solution provided by the ES in order to enhance user understanding, trust, and confidence on the ES.

Knowledge Base

The Knowledge base contains

1. **Facts:** statements of truth in the application domain. Represented as declarative statements.
e.g., A car requires fuel to move.
2. **Procedural rules:** well-defined, invariant rules. Expressed as "if-then" statements or algorithms describing a sequence of steps to achieve a goal.
e.g., - If the car won't start and the battery voltage is below 12V, then recharge or replace the battery.
- Procedure to prepare tea
3. **Heuristic rules:** procedures to be followed when invariant procedural rules are not available. Represented as "rules of thumb" derived from experience rather than hard facts.
e.g., If a computer is running slow, try restarting before looking for more complex issues.

Inference engine

- It is a control mechanism which attempts to arrive at a solution using the KB & the current state information.
- The “current state” is a dynamic store of info. which is:
 - ✓ Either known as facts from the beginning,
 - ✓ Supplied by the user during processing, or
 - ✓ Deduced by the inference engine.

Production Rules

- Knowledge base is also called as a production system.
- Production rules can be expressed in IF-THEN format.
- In rule-based systems, the inference engine determines the applicability of a rule (LHS of which rule is satisfied by the facts.)

Example Rules

IF ... THEN Rules

Rule: Red_Light

IF the light is red

THEN stop

Rule: Green_Light

IF the light is green

THEN go

antecedent
(left-hand-side)

consequent
(right-hand-side)

Production Rules

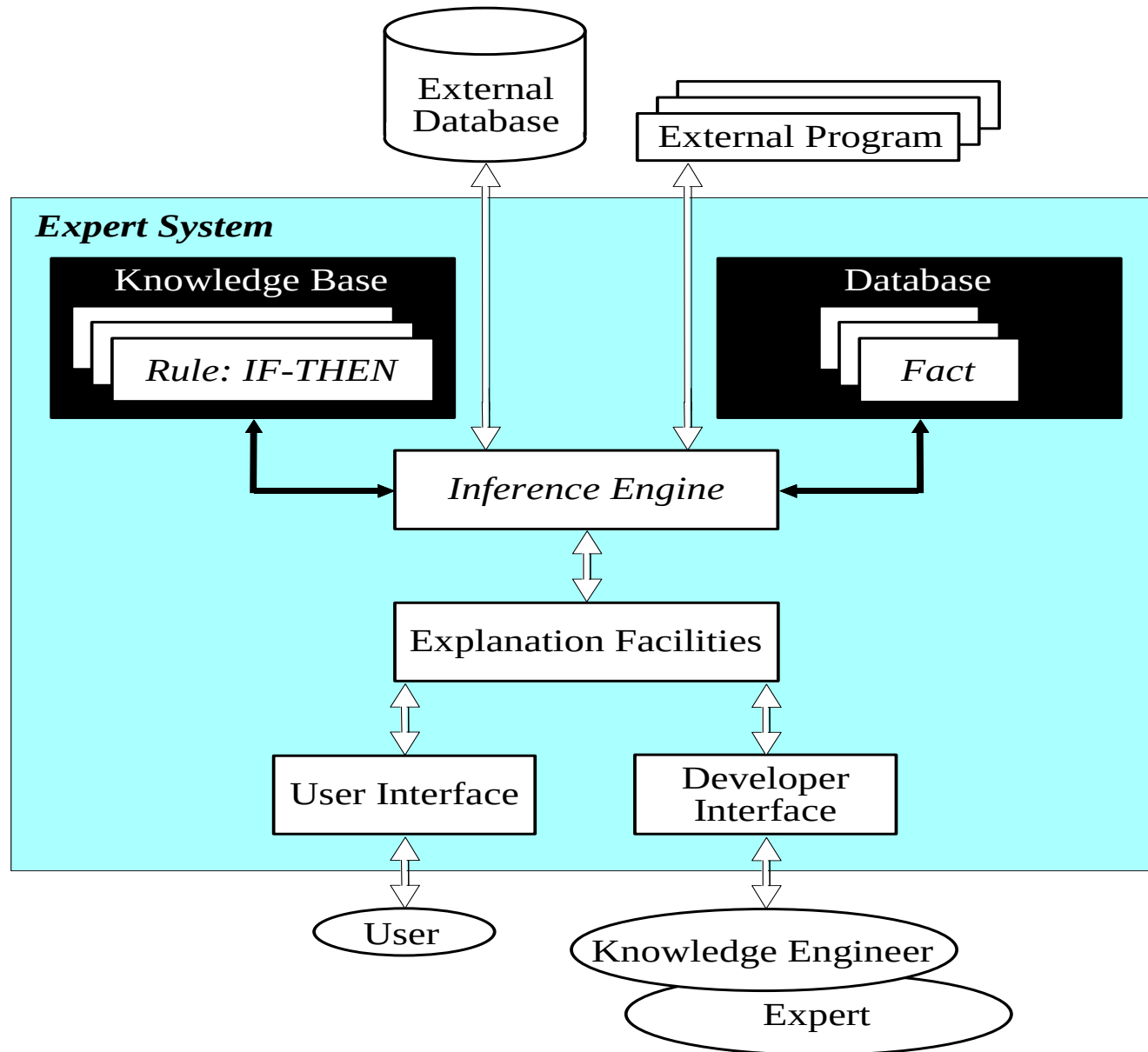
the light is red ==> stop

the light is green ==> go

antecedent (left-hand-side)

consequent
(right-hand-side)

Rule-based expert system



General Methods of Inferencing

1. Forward chaining (data-driven): starts with given facts and use rules to draw conclusions/take actions.

The Process

1. Reasoning begins from a “start” state.
2. Rules are selected by matching the L.H.S., i.e., Condition part of the rules are matched against the state description.
3. The R.H.S. of the matched rule is added to the updated state description.
4. The process is repeated until a useful conclusion is made.

This process is known as “Data-driven” because input data is used to guide the inferencing process. It follows a “**recognize-act**” cycle.

Suitable for prognosis, monitoring, and control applications.

Examples: CLIPS, OPS5

Example:

R1: Encouraged (stud) -> Motivated (stud)

R2: Motivated (stud) & Healthy (Stud) ->
Work-hard (stud)

R3: Work-hard (stud) & Has-ambition (stud) ->
Good (stud)

R4: Good (stud) -> Succeed (stud)

When a student is encouraged, is healthy & has ambition, the student will succeed.

2. Backward chaining (goal-driven)

It starts with a hypothesis (goal) and look for rules/facts that support the hypothesis.

The Process

1. Reasoning begins with a “goal” state (goal to be achieved).
2. Rules are selected by matching the R.H.S.
3. The L.H.S. of a selected rule becomes the next sub-goal to be achieved.
4. The process continues till facts are found to match the lowest sub-goal **else the goal is unachievable**.

Suitable for diagnosis problems

e.g. PROLOG, MYCIN

Rule Editor

Function of Rule editor:

- ✓ addition of rules.
- ✓ deletion of rules.
- ✓ modification of rules.

Characteristics of Rule editor:

- ✓ Ability to check for consistency when new rules are added.
- ✓ Add missing conditions to a rule.
- ✓ Rule validation & verification
- ✓ Locate redundant /unnecessary rules.